CLOUD & DEVOPS ENGINEERING

4-MONTH PROFESSIONAL TRAINING CURRICULUM

16 weeks • Practical labs • Topic projects • Final capstone

Prepared as a training curriculum aligned to the Cloud & DevOps Academy flyer supplied by the requester.

PROGRAMME OVERVIEW

- Duration: 4 months (16 weeks).
- Delivery: Recommended 3 instructor-led sessions per week, supported by guided labs and independent practice.
- Approach: Approximately 70% hands-on practice and 30% instruction, discussion, review and assessment.
- Audience: Beginners with basic computer skills, IT students, developers and aspiring cloud/DevOps engineers.
- Prerequisite: Basic Linux knowledge is recommended; provide a short orientation for learners who need it.
- Outcome: Portfolio demonstrating infrastructure automation, CI/CD, containers, Kubernetes, monitoring, logging and GitOps.

LEARNING OUTCOMES

- Administer Linux and troubleshoot networking and services.
- Collaborate using Git/GitLab and implement CI/CD pipelines.
- Build and secure Docker images and containerized applications.
- Provision AWS resources using secure cloud practices and Terraform.
- Automate configuration with Ansible.
- Deploy and operate applications on Kubernetes.
- Monitor with Prometheus/Grafana and centralize logs with ELK.
- Implement GitOps with Argo CD and integrate DevSecOps checks.

MONTH 1 — LINUX, NETWORKING, GIT & CLOUD FOUNDATIONS

Week 1: Linux Administration and Shell Fundamentals

Topics and practical skills

- Linux distributions, terminal navigation, filesystem hierarchy, files and directories
- Users, groups, permissions, ownership, sudo and access control
- Package management, processes, services, systemd, logs and disk usage
- Bash basics: variables, pipes, redirection, grep, find, awk, sed and shell scripts

Projects / deliverables

- Build and document a Linux server baseline; create users and groups with least privilege.
- Write a Bash script that checks disk space, memory, uptime and key service status and writes a timestamped report.

Week 2: Networking, SSH and Troubleshooting

Topics and practical skills

- IP addressing, CIDR/subnets, DNS, DHCP, routing, ports and common protocols
- HTTP/HTTPS, TLS basics, firewalls, security groups concept and network troubleshooting
- SSH keys, secure remote access, scp/rsync, curl, ping, traceroute, dig/nslookup, ss/netstat
- Environment variables, secrets hygiene, basic system hardening and patching

Projects / deliverables

- Set up two Linux VMs and configure key-based SSH access.
- Troubleshoot a deliberately broken web service using logs, DNS, ports and firewall checks; submit a troubleshooting runbook.

Week 3: Git, GitLab and Collaboration

Topics and practical skills

- Git repositories, commits, branches, merges, tags, reset/revert and conflict resolution
- GitLab projects, issues, merge requests, code review and protected branches
- README writing, branching strategy, semantic versioning and release notes
- Basic YAML, application configuration and team workflows

Projects / deliverables

- Create a team GitLab project with feature branches, merge requests, reviews, issues and tagged releases.
- Publish a small web application with a clear README, branching policy and release notes.

Week 4: Cloud Concepts and AWS Fundamentals

Topics and practical skills

- Cloud service models and deployment models; regions, Availability Zones and shared responsibility
- AWS IAM users/roles/policies, MFA, least privilege and AWS CLI profiles
- EC2, EBS, S3, VPC, subnets, route tables, internet gateways and security groups
- AWS billing, budgets, tagging, CloudTrail basics and cost-aware lab cleanup

Projects / deliverables

- Deploy a basic web server on EC2 in a properly configured VPC and restrict inbound access.
- Host a static website on S3, configure appropriate access, and document security and cost decisions.

MONTH 2 — DOCKER, AWS SERVICES, TERRAFORM & ANSIBLE

Week 5: Docker and Containerization

Topics and practical skills

- Containers versus virtual machines; images, registries, layers and Docker architecture
- Dockerfile instructions, build context, image tags and multi-stage builds
- Containers, ports, volumes, networks, environment variables and health checks
- Docker Compose, container logs, resource limits and image scanning concepts

Projects / deliverables

- Containerize a simple application and database using Docker Compose.
- Create an optimized multi-stage image, persist data with volumes and document how to run and troubleshoot it.

Week 6: AWS Application Services and Secure Architecture

Topics and practical skills

- EC2 launch templates and Auto Scaling concepts; Application Load Balancer
- S3 lifecycle/versioning, IAM roles, Lambda fundamentals and event-driven patterns
- CloudWatch metrics, logs and alarms; S3 static website hosting and access models
- VPC design, private/public subnets, NAT concept, Secrets Manager/Systems Manager overview

Projects / deliverables

- Deploy a highly available sample web application behind a load balancer, using health checks and least-privilege IAM.
- Build a small Lambda function triggered by an event and capture logs/metrics; set a CloudWatch alarm.

Week 7: Terraform Infrastructure as Code

Topics and practical skills

- IaC principles, providers, resources, data sources, variables, outputs and locals
- Terraform init/validate/plan/apply/destroy workflow and state management
- Modules, remote state concepts, state locking, workspaces and environment separation
- Secrets handling, plan review, drift detection, tagging and cost controls

Projects / deliverables

- Use Terraform to provision a VPC, subnets, security groups, an EC2 instance and an S3 bucket.
- Refactor infrastructure into reusable modules, parameterize dev/test environments and demonstrate safe teardown.

Week 8: Ansible Configuration Management

Topics and practical skills

- Inventory, ad hoc commands, modules, facts and playbooks
- Variables, handlers, templates, roles, loops, conditionals and vault/encrypted variables
- Idempotency, SSH connectivity, privilege escalation and reusable automation
- Terraform versus Ansible; infrastructure provisioning versus configuration management

Projects / deliverables

- Write an Ansible role that installs and configures Nginx, deploys a web page and enables a firewall rule.
- Combine Terraform and Ansible to create a VM and configure it automatically; verify repeat runs are idempotent.

MONTH 3 — CI/CD, KUBERNETES & DEVSECOPS

Week 9: CI/CD with GitLab Pipelines

Topics and practical skills

- CI/CD concepts, pipeline stages, jobs, runners, artifacts, caches and variables
- GitLab CI YAML, rules, dependencies, environments and manual approvals
- Build, test, package and publish Docker images to a registry
- Deployment strategies: rolling, blue/green and canary concepts; rollback and release gates

Projects / deliverables

- Build a pipeline that lints, tests, builds and pushes a container image on merge to the main branch.
- Deploy to a test environment after a successful pipeline and demonstrate a rollback after a failed release.

Week 10: Kubernetes Core Concepts

Topics and practical skills

- Cluster architecture: control plane, nodes, API server, scheduler and etcd concepts
- Pods, Deployments, ReplicaSets, Services, namespaces, labels and selectors
- ConfigMaps, Secrets, probes, resource requests/limits and rolling updates
- kubectl, YAML manifests, local cluster using kind or Minikube; troubleshooting events and logs

Projects / deliverables

- Deploy a multi-replica web application to a local Kubernetes cluster with Service, ConfigMap and health probes.
- Break a deployment intentionally and diagnose it using kubectl describe, logs, events and rollout history.

Week 11: Kubernetes Networking, Storage and Operations

Topics and practical skills

- Ingress and ingress controllers, service discovery and network policy concepts
- PersistentVolumes, PersistentVolumeClaims, StorageClasses and stateful workloads
- Horizontal Pod Autoscaler concepts, requests/limits, scheduling and availability
- Helm charts, values files, release upgrades and rollback; Kubernetes security contexts and RBAC

Projects / deliverables

- Package an application as a Helm chart with configurable replicas, resources and environment values.
- Deploy a stateful component with persistent storage and configure RBAC for a restricted service account.

Week 12: DevSecOps and Pipeline Security

Topics and practical skills

- Secure SDLC, threat modeling basics and security checks in CI/CD
- Secret scanning, dependency scanning, SAST/DAST concepts and software bill of materials
- Container image vulnerability scanning, minimal base images and non-root containers
- Terraform/IaC scanning, policy checks, artifact provenance, approvals and supply-chain security

Projects / deliverables

- Add automated code, dependency, container and IaC checks to the GitLab pipeline; document and fix sample findings.
- Harden a containerized app: run as non-root, remove embedded secrets, pin image versions and create a remediation report.

MONTH 4 — OBSERVABILITY, GITOPS & CAPSTONE

Week 13: Monitoring with Prometheus and Grafana

Topics and practical skills

- Metrics, logs and traces; golden signals, SLIs/SLOs and alert design
- Prometheus scraping, exporters, labels, PromQL basics and alert rules
- Grafana data sources, dashboards, panels, variables and alert notifications
- Application and infrastructure monitoring; avoiding noisy alerts and dashboard overload

Projects / deliverables

- Instrument a sample application and Linux host, collect metrics with Prometheus and build a Grafana dashboard.
- Create alerts for high CPU, low disk space and application unavailability; test and document alert response.

Week 14: Centralized Logging with ELK Stack

Topics and practical skills

- Structured logging, log levels, correlation IDs and useful event fields
- Elasticsearch indexes and mappings; Kibana Discover, visualizations and dashboards
- Log ingestion using Elastic Agent or Filebeat/Logstash where appropriate
- Retention, access control, sensitive-data redaction and troubleshooting workflows

Projects / deliverables

- Collect application and system logs into the ELK Stack and build a Kibana dashboard.
- Create a log-based investigation exercise that identifies an application error and documents root cause.

Week 15: GitOps with Argo CD and Release Engineering

Topics and practical skills

- GitOps principles, desired state, reconciliation and declarative delivery
- Argo CD applications, repositories, sync policies, health status and drift detection
- Separate application source from deployment manifests; environment promotion and secrets strategy
- Progressive delivery concepts, release approvals, rollback, backup and operational runbooks

Projects / deliverables

- Deploy a Kubernetes application through Argo CD from a Git repository; demonstrate sync and drift correction.
- Promote a release from staging to production-like namespace using reviewed Git changes and demonstrate rollback.

Week 16: Capstone Integration, Review and Demo Day

Topics and practical skills

- Architecture review, reliability, security, cost controls and documentation
- End-to-end pipeline review, incident simulation, troubleshooting and handover
- Portfolio preparation, technical presentation, peer review and interview practice

Projects / deliverables

- CAPSTONE: Build a secure, observable cloud-native application delivery platform.
- Provision AWS infrastructure with Terraform; configure hosts with Ansible where needed; containerize an application; build GitLab CI/CD; deploy to Kubernetes/EKS or a local Kubernetes cluster if AWS budget is limited; manage delivery with Argo CD; monitor with Prometheus/Grafana; centralize logs with ELK; add security checks and documentation.
- Submit architecture diagram, source repository, infrastructure code, pipeline, Kubernetes/Helm manifests, dashboards, runbook, cost estimate, security checklist and a live demonstration.

RECOMMENDED TOOL STACK

- Linux (Ubuntu), Bash, SSH and networking utilities
- Git and GitLab
- Docker and Docker Compose
- AWS: IAM, VPC, EC2, S3, Lambda, CloudWatch, load balancing and EKS concepts
- Terraform
- Ansible
- GitLab CI/CD and container registry
- Kubernetes, kubectl, Helm, kind or Minikube; EKS where budget allows
- Prometheus and Grafana
- Elasticsearch, Logstash and Kibana (ELK)
- Argo CD
- Security tooling concepts: secret/dependency/image/IaC scanning

FINAL CAPSTONE: SECURE CLOUD-NATIVE DELIVERY PLATFORM

Students work individually or in small teams to design, automate, deploy, secure and observe a production-style application such as a task manager, inventory service, student portal or API.

- Application source code in GitLab with documented branching and merge-request workflow.
- Terraform-managed cloud infrastructure with least-privilege access, tags and cost notes.
- Automated CI pipeline for validation, tests, image build, image scan and registry push.
- Kubernetes deployment using Helm or manifests, health probes, resource requests/limits and RBAC.
- Argo CD GitOps application sync and release/rollback evidence.
- Prometheus/Grafana dashboards and tested alert rules.
- Centralized logs and Kibana investigation dashboard.
- Security findings/remediation report, architecture diagram, README and operations runbook.

ASSESSMENT AND GRADING

Assessment	Weight	Evidence
Weekly practical labs	30%	Hands-on completion, troubleshooting and evidence
Topic projects and code reviews	20%	Working deliverables, documentation and Git hygiene
Mid-program practical assessment	15%	Docker, AWS, Terraform, Ansible and CI/CD
DevSecOps and observability assessment	10%	Security checks, dashboards, logs and alerts
Final capstone project	25%	Architecture, automation, security, reliability and demo

SUGGESTED WEEKLY DELIVERY PATTERN

- Session 1 — concepts, instructor demonstration and guided notes.
- Session 2 — instructor-led lab with troubleshooting checkpoints.
- Session 3 — project workshop, code review, learner presentation and quiz.
- Independent work — 2–4 hours per week for practice, documentation and portfolio updates.

TRAINER AND LAB SETUP NOTES

- Each learner should have a laptop with at least 8 GB RAM; 16 GB is recommended for local containers and Kubernetes labs.

- Use a Linux VM, WSL2 or dedicated Linux machine. Provide GitLab access and a container registry.
- Use AWS learner accounts with MFA, budget alerts, least-privilege permissions and clear cleanup instructions.
- AWS EKS and always-on cloud labs can incur costs. Use kind/Minikube for core Kubernetes exercises and AWS for selected integration labs if budgets are limited.
- Never commit credentials or secrets to Git. Use temporary lab credentials and approved secret-management methods.
- Require evidence for each project: repository, README, screenshots/logs where useful, architecture diagram and short demo.
- Require safe teardown of cloud resources at the end of each lab to avoid unexpected charges.

COMPLETION REQUIREMENTS

- Complete at least 80% of labs and weekly assignments.
- Submit topic projects and pass the practical mid-program assessment.
- Demonstrate the final capstone and submit source code, infrastructure/deployment files, documentation and runbook.
- Explain security decisions, failure recovery, monitoring signals and cost-control choices during the final review.